Software Quality Standard - (SQS)

Software Quality Standard - (SQS)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Parent Standard

Governed Space: Software Quality Governance

Standard ID: SLGA-SQS-005

Origin ID: OOF.PS.SLGA.05.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 12.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Software Quality Standard (SQS) governs the definition, establishment,

assessment, verification, validation, acceptance, and lifecycle continuity of software quality conditions.

SQS establishes the methodological conditions under which software quality

can be recognized as an identifiable, evidence-supported, context-dependent governance state rather than an assumption derived from successful execution, completed testing, favorable performance indicators, or declared compliance with quality requirements.

It governs the relationships among software quality requirements, quality

criteria, software characteristics, testing and verification activities, observed software behavior, identified defects, quality acceptance decisions, and the evidence supporting quality determinations.

SQS distinguishes declared quality from demonstrated quality, test

execution from verified software behavior, defect identification from defect resolution, and quality acceptance from unconditional software correctness.

It applies to software developed, maintained, tested, assessed, or operated

by humans, organizations, automated development systems, AI coding assistants, autonomous software agents, and software infrastructure mechanisms.

The standard is technology-independent and applies across software development methods, programming languages, application domains, deployment environments, and lifecycle models.

Its principal output is:

Governed Software Quality State

Its foundational distinction is:

Declared Software Quality ≠ Demonstrated Software Quality.

Operational Role

The operational role of SQS is:

To ensure that software quality remains explicitly defined, assessed against competent criteria, supported by verifiable evidence, bounded by known limitations, and traceable to the software artifact and operational context actually evaluated throughout the software lifecycle.

SQS establishes the quality-governance layer between software configuration

and software security within SLGA™.

It receives relevant governed requirements, creation, dependency, and

configuration states.

It establishes whether material software quality claims are sufficiently supported by appropriate criteria, assessment methods, observations, defect evidence, and acceptance decisions.

It does not independently determine software security, authorize software release or deployment, replace software requirements governance, or guarantee defect-free operation.

Core Question

How can software quality be defined, demonstrated, assessed, and accepted through competent evidence while ensuring that successful tests, favorable metrics, or declared compliance are not mistaken for proof that software is sufficiently fit for its intended purpose and operating conditions?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

SQS governs the fifth principal stage:

Software Quality

Its principal upstream references include:

Governed Software Requirements Baseline

From Software Requirements Standard (SRS).

Governed Software Creation State

From Software Creation Standard (SCS).

Governed Software Dependency State

From Software Dependency Standard (SDS).

Governed Software Configuration State

From Software Configuration Standard (SCFS).

Its principal output is:

Governed Software Quality State

This output supports:

Software Security Standard (SSS)

Software Release Standard (SRLS)

Software Deployment Standard (SDPS)

Software Change Management Standard (SCMS)

Software Retirement Standard (SRTS)

and cross-cutting governance under:

Software Lifecycle Control Standard (SLCS)

SQS may be invoked repeatedly whenever software requirements, artifacts, dependencies, configuration, intended use, observed behavior, or material quality evidence changes.

• Governed Space

Software Quality Governance

This space governs the conditions under which software quality claims,

assessments, defects, and acceptance decisions remain identifiable, evidence-supported, and lifecycle-relevant.

Its principal dimensions include:

• Software quality governance object identity

• Software quality purpose

• Software quality scope

• Software quality characteristics

• Quality requirements

• Quality criteria

• Quality thresholds

• Quality acceptance conditions

• Quality assessment context

• Intended software use

• Software artifact identity

• Software version correspondence

• Configuration correspondence

• Quality evidence

• Evidence competence

• Evidence integrity

• Test design

• Test coverage

• Test execution

• Test environment correspondence

• Observed software behavior

• Expected–observed behavior comparison

• Quality measurement

• Measurement limitations

• Defect identification

• Defect classification

• Defect severity

• Defect impact

• Defect attribution

• Defect correction references

• Defect verification

• Regression assessment

• Quality acceptance

• Conditional acceptance

• Residual quality limitations

• Quality uncertainty

• Quality conflicts

• Quality evidence gaps

• Critical quality gaps

• Quality continuity

• Quality reassessment

• Quality governance states

Software quality governance shall remain distinguishable from software

security, operational safety, legal compliance, and commercial suitability.

Software Quality Governance Object

Every material SQS assessment shall establish a Software Quality Governance

Object.

The object represents an identifiable software quality assessment and its

applicable governance conditions.

It should identify:

Software Object: The software being assessed.

Artifact Identity: The relevant artifact or software version.

Quality Purpose: The reason quality is being assessed.

Quality Scope: The boundaries of the assessment.

Intended Use: The purpose and operating context against which quality is evaluated.

Quality Criteria: The conditions against which software is assessed.

Assessment Methods: The methods used to evaluate quality.

Assessment Environment: The relevant evaluation conditions.

Observed Results: The evidence obtained.

Defect State: The identified and unresolved defect conditions.

Acceptance State: The recognized quality acceptance condition.

Governance State: The overall quality determination.

A software quality governance object shall remain distinguishable from a

test report, quality score, defect list, or certification claim.

Software Quality Governance Object ≠ Software Test Report.

Quality Governance Principles

SQS establishes the following principles. 7.1 Quality Must Be Defined

Software quality cannot be meaningfully assessed without identifying the

criteria against which it is evaluated. 7.2 Quality Must Be Contextualized

Quality determinations shall remain connected to intended software use,

relevant operating conditions, and applicable requirements. 7.3 Quality Must Be Evidence-Supported

Declared quality shall not substitute for competent observations,

measurements, verification, or validation. 7.4 Quality Evidence Must Be Bounded

Evidence shall support only the software objects, conditions, and claims

actually assessed. 7.5 Quality Acceptance Must Be Distinguishable

Acceptance decisions shall remain separate from the evidence on which they

rely. 7.6 Quality Is Not Absolute

A software product may satisfy defined criteria while retaining defects, limitations, and unassessed behavior. 7.7 Quality Must Remain Reassessable

Material changes to software, configuration, dependencies, intended use, or

operating conditions may require renewed quality assessment.

Software Quality Purpose

SQS shall identify why software quality is being assessed.

Purposes may include:

• Development validation

• Requirements conformity

• Functional verification

• Reliability assessment

• Performance assessment

• Compatibility evaluation

• Maintainability assessment

• Release preparation

• Operational readiness support

• Defect correction verification

• Regression evaluation

• Lifecycle maintenance

Quality purpose shall remain distinguishable from demonstrated quality.

Quality Assessment Purpose ≠ Quality Assessment Result.

Software Quality Scope

SQS shall establish the boundaries of material quality assessments.

Scope may concern:

• Individual functions

• Software components

• Integrated applications

• Complete software products

• Specific artifacts

• Particular versions

• Defined environments

• Intended usage scenarios

• Relevant operating conditions

Scope shall distinguish assessed, excluded, and unresolved software

behavior.

Tested Software Scope ≠ Complete Software Behavior.

Software Quality Characteristics

SQS shall identify the quality characteristics relevant to the software and

its intended use.

Characteristics may include:

• Functional suitability

• Reliability

• Performance efficiency

• Compatibility

• Usability and interaction quality

• Maintainability

• Portability and adaptability

• Recoverability

• Relevant operational robustness

Quality characteristics shall be selected according to the governed

software reality rather than imposed as an identical fixed set on every software object.

Security-specific requirements remain governed by SSS, with relevant

quality relationships preserved.

Quality Characteristic Selection ≠ Quality Characteristic Satisfaction.

Quality Requirements Correspondence

SQS shall preserve the relationship between software quality criteria and

applicable governed requirements.

Relevant relationships may include:

• Functional requirements

• Non-functional requirements

• Performance expectations

• Reliability conditions

• Compatibility constraints

• Usability expectations

• Maintainability requirements

• Acceptance conditions

Requirements definition and authorization remain governed by SRS.

SQS governs their quality-assessment correspondence.

Requirement Existence ≠ Demonstrated Requirement Satisfaction.

Quality Criteria Governance

SQS shall establish sufficiently identifiable criteria for assessing

material quality claims.

Criteria should identify:

• The quality characteristic

• The assessed software object

• The applicable condition

• The expected result

• The accepted measurement or observation method

• Relevant tolerances

• The evidence required

Criteria shall remain distinguishable from results.

Defined Quality Criterion ≠ Satisfied Quality Criterion.

Quality Threshold Governance

SQS shall identify applicable quality thresholds where measurable conditions are relevant.

Thresholds may concern:

• Response time

• Error rate

• Resource consumption

• Reliability

• Availability

• Test coverage

• Defect severity

• Recovery performance

• Compatibility

A numerical threshold shall remain associated with its measurement method

and operating context.

Quality Threshold Achievement ≠ Complete Software Quality.

Quality Acceptance Conditions

SQS shall identify the conditions under which material quality results may be accepted.

Acceptance conditions may include:

• Required assessment completion

• Satisfied quality criteria

• Sufficient evidence

• Defined defect tolerances

• Competent acceptance authority

• Explicit residual limitations

• Required reassessment conditions

Acceptance conditions shall remain distinguishable from acceptance

decisions.

Quality Acceptance Criteria ≠ Quality Acceptance Decision.

Intended Use Correspondence

SQS shall assess whether quality criteria and evidence correspond to the

software's intended use.

A software product may function correctly under one usage pattern while being unsuitable for another.

Relevant dimensions may include:

• User activity

• Workload

• Operational environment

• Data characteristics

• Integration conditions

• Expected service behavior

Technical Functionality ≠ Fitness for Intended Use.

Software Artifact Correspondence

SQS shall identify the software artifact or version to which quality

evidence applies.

Relevant information may include:

• Artifact identity

• Version reference

• Content identity

• Build reference

• Relevant configuration state

Quality evidence associated with one artifact shall not automatically

establish the quality of a different artifact.

Tested Artifact ≠ Every Artifact Bearing the Same Version Label.

Configuration Correspondence

SQS shall preserve the relationship between quality assessment and the

software configuration actually evaluated.

Relevant dimensions may include:

• Configuration baseline

• Runtime parameters

• Component versions

• Build settings

• Test environment

• Relevant external services

A successful assessment under one configuration shall not automatically

establish equivalent quality under another.

Tested Configuration ≠ Every Operational Configuration.

Quality Assessment Method Governance

SQS shall identify the methods used to assess software quality.

Methods may include:

• Static analysis

• Dynamic testing

• Functional testing

• Integration testing

• System testing

• Performance testing

• Reliability testing

• Compatibility testing

• Usability assessment

• Code review

• Formal verification where applicable

• Operational observation

Methods shall be appropriate to the quality claims they support.

Assessment Method Execution ≠ Competent Quality Assessment.

Assessment Method Suitability

SQS shall assess whether the selected method can meaningfully evaluate the relevant quality characteristic.

For example:

A unit test may assess a defined component behavior.

It may not establish end-to-end system reliability.

A performance benchmark may assess a defined workload.

It may not establish performance under every operational workload.

Successful Assessment Method ≠ Universal Quality Evidence.

Test Design Governance

SQS shall govern the quality-related conditions under which test cases and

test scenarios are defined.

Test design should identify:

• Tested object

• Test purpose

• Relevant requirement

• Input conditions

• Expected behavior

• Execution environment

• Evaluation criterion

• Relevant limitations

Test design shall remain distinguishable from test execution.

Defined Test Case ≠ Executed Test Case.

Test Coverage Governance

SQS shall establish the scope and limitations of material test coverage.

Coverage may concern:

• Requirements

• Functions

• Code paths

• Branches

• Components

• Interfaces

• Usage scenarios

• Failure conditions

• Environment combinations

Coverage measurements shall remain associated with their definitions.

High Test Coverage ≠ Demonstrated Software Correctness.

Test Execution Governance

SQS shall preserve material evidence concerning the execution of quality

assessments.

Evidence may include:

• Test identity

• Software artifact

• Configuration

• Test inputs

• Execution environment

• Execution time

• Observed results

• Execution deviations

Successful test execution shall remain distinguishable from successful test

outcomes.

Test Execution Completion ≠ Test Success.

Test Environment Correspondence

SQS shall assess whether the test environment is suitable for the intended quality claim.

Relevant conditions may include:

• Platform

• Software dependencies

• Runtime configuration

• Workload

• Network conditions

• Data characteristics

• External services

• Hardware constraints

A test environment need not always be identical to production, but

differences material to the quality claim shall remain explicit.

Test Environment Similarity ≠ Demonstrated Operational Equivalence.

Expected Software Behavior

SQS shall establish the expected behavior relevant to material quality

assessments.

Expected behavior may derive from:

• Governed requirements

• Quality criteria

• Interface contracts

• Defined usage scenarios

• Accepted behavioral specifications

Expected behavior shall remain distinguishable from actual observed

behavior.

Specified Behavior ≠ Observed Behavior.

Observed Software Behavior

SQS shall preserve competent evidence of software behavior during

assessment.

Observation may involve:

• Functional outputs

• Error conditions

• Execution traces

• Response times

• Resource consumption

• Interface behavior

• Recovery behavior

• Relevant operational effects

Observed behavior shall remain connected to the assessed artifact and

conditions.

Software Execution ≠ Verified Software Behavior.

Expected–Observed Behavior Correspondence

SQS shall assess the relationship between expected and observed software

behavior.

Assessment may identify:

• Correspondence

• Deviation

• Partial correspondence

• Unobserved conditions

• Inconclusive results

• Material limitations

Correspondence shall remain bounded by the evaluated scenarios and evidence.

Observed Expected Output ≠ Complete Behavioral Correctness.

Functional Quality Governance

SQS shall govern evidence supporting claims that software performs defined functions under applicable conditions.

Functional quality assessment may concern:

• Correct outputs

• Required operations

• Interface behavior

• Error handling

• Relevant boundary conditions

• Defined input variations

Functional success in a limited scenario shall not establish complete

functional suitability.

Functional Test Success ≠ Complete Functional Suitability.

Reliability Quality Governance

SQS shall govern evidence supporting material software reliability claims.

Assessment may concern:

• Failure frequency

• Stability

• Fault tolerance

• Recovery behavior

• Sustained operation

• Defined workload conditions

Reliability evidence shall remain associated with the observed period and

conditions.

Observed Stability ≠ Guaranteed Future Reliability.

Performance Quality Governance

SQS shall govern evidence supporting software performance claims.

Assessment may concern:

• Latency

• Throughput

• Resource efficiency

• Scalability

• Workload response

• Performance degradation

Performance measurements shall preserve workload, environment, and method

context.

Favorable Benchmark Result ≠ Universal Performance Quality.

Compatibility Quality Governance

SQS shall govern evidence supporting software compatibility claims.

Compatibility may concern:

• Operating systems

• Hardware platforms

• Software versions

• Interfaces

• Data formats

• Integration environments

Compatibility evidence shall remain bounded by the assessed combinations.

Compatibility with One Environment ≠ Universal Compatibility.

Usability and Interaction Quality Governance

SQS shall recognize relevant software usability and interaction quality

conditions.

Assessment may concern:

• Task completion

• Interface clarity

• Interaction consistency

• Accessibility requirements

• Error prevention

• User feedback

• Recoverability from interaction errors

Usability quality shall remain contextualized to intended users and tasks.

Interface Functionality ≠ Demonstrated Usability.

Maintainability Quality Governance

SQS shall recognize relevant software maintainability characteristics.

Assessment may concern:

• Modularity

• Changeability

• Analyzability

• Testability

• Complexity

• Documentation correspondence

• Relevant maintenance constraints

Maintainability claims shall be supported by competent criteria and

evidence.

Readable Code ≠ Demonstrated Maintainability.

Portability and Adaptability Quality Governance

SQS shall govern evidence supporting software portability and adaptability

claims.

Assessment may concern:

• Installation

• Platform migration

• Environment adaptation

• Configuration flexibility

• Compatibility with target conditions

Portability claims shall remain bounded by assessed platforms and

environments.

Portable Design Intent ≠ Verified Portability.

Quality Measurement Governance

SQS shall identify material measurements used to support software quality

determinations.

Measurements may include:

• Test pass rates

• Defect density

• Response times

• Reliability indicators

• Coverage measurements

• Resource utilization

• Maintainability indicators

Each measurement shall preserve its definition, method, scope, and

limitations.

Quality Metric ≠ Software Quality State.

Measurement Interpretation

SQS shall assess whether quality measurements support the conclusions drawn

from them.

For example:

A high test pass rate may indicate that the executed tests passed.

It does not establish that all relevant behaviors were tested.

A low observed defect count may indicate that few defects were detected.

It does not establish that few defects exist.

Measured Result ≠ Unqualified Quality Conclusion.

Quality Evidence Governance

SQS shall preserve evidence supporting material quality claims.

Evidence may include:

• Requirements correspondence

• Test plans

• Test execution records

• Assessment results

• Quality measurements

• Defect records

• Verification results

• Validation records

• Acceptance decisions

Evidence shall remain associated with the specific software object,

condition, and claim it supports.

Quality Evidence Presence ≠ Quality Claim Validity.

Quality Evidence Competence

SQS shall assess whether evidence is competent to support the relevant quality determination.

A test report may support the results of executed tests.

A benchmark may support performance under defined conditions.

A defect record may support the existence of an observed issue.

An acceptance record may support a competent decision.

These records establish different facts.

Evidence of Testing ≠ Evidence of Complete Quality.

Quality Evidence Integrity

SQS shall preserve the integrity of material quality evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Artifact correspondence

• Configuration correspondence

• Temporal relevance

• Method suitability

• Material contradictions

• Preservation history

An intact quality report shall not independently establish the validity of

its conclusions.

Intact Quality Report ≠ Verified Software Quality.

Defect Identification Governance

SQS shall establish identifiable records for material software defects.

A defect record should identify:

• Affected software object

• Relevant artifact and configuration

• Observed behavior

• Expected behavior

• Reproduction conditions

• Supporting evidence

• Assessment status

A reported defect shall remain distinguishable from a verified defect.

Reported Software Defect ≠ Verified Software Defect.

Defect Classification

SQS shall recognize material defect classifications.

Classifications may concern:

• Functional defect

• Reliability defect

• Performance defect

• Compatibility defect

• Usability defect

• Maintainability-related defect

• Environment-specific defect

Security vulnerabilities remain subject to SSS, with relevant quality

relationships preserved.

Defect Classification ≠ Defect Severity.

Defect Severity Governance

SQS shall assess the material severity of identified defects within the relevant software context.

Severity may depend on:

• Functional impact

• Affected users or systems

• Frequency

• Recoverability

• Data integrity implications

• Operational consequences

• Applicable quality requirements

Severity shall remain distinguishable from remediation priority.

Defect Severity ≠ Defect Remediation Priority.

Defect Impact Governance

SQS shall preserve material evidence concerning the consequences of

identified defects.

Impact may concern:

• Software functionality

• Service continuity

• Data processing

• User interaction

• Performance

• Integration behavior

• Maintenance

Impact assessment shall remain bounded by available evidence.

Potential Defect Impact ≠ Observed Defect Impact.

Defect Attribution

SQS shall distinguish observed defects from claims concerning their causes.

Potential causes may include:

• Implementation errors

• Incorrect requirements interpretation

• Dependency behavior

• Configuration mismatch

• Environment conditions

• Integration defects

• Data-related conditions

Attribution shall remain evidence-supported.

Defect Observation ≠ Verified Root Cause.

Defect Correction References

SQS shall preserve relationships between identified defects and relevant

corrective actions.

Correction references may include:

• Proposed correction

• Code change

• Configuration adjustment

• Dependency modification

• Workaround

• Replacement artifact

Execution of corrective changes remains subject to competent creation,

configuration, and change-management governance.

Defect Correction Action ≠ Verified Defect Resolution.

Defect Resolution Verification

SQS shall assess whether a defect has been sufficiently resolved within the defined scope.

Verification may involve:

• Reproduction of the original failure

• Execution of corrective tests

• Expected–observed behavior comparison

• Relevant artifact correspondence

• Configuration correspondence

• Regression assessment

A defect closure record shall remain distinguishable from verified

resolution.

Closed Defect Ticket ≠ Verified Defect Resolution.

Regression Quality Governance

SQS shall assess material quality effects following software changes.

Regression assessment may concern:

• Previously functioning behavior

• Related software components

• Interfaces

• Performance

• Reliability

• Relevant usage scenarios

Passing selected regression tests shall not establish that no regression exists elsewhere.

Regression Test Success ≠ Absence of Regression Risk.

Quality Validation

SQS shall determine whether the software satisfies the defined quality

conditions for its intended use within the assessed scope.

Validation may consider:

• Applicable quality requirements

• Quality criteria

• Intended use

• Artifact identity

• Configuration

• Test evidence

• Observed behavior

• Defect conditions

• Material limitations

Validation shall preserve uncertainty and exclusions.

Validated Quality Scope ≠ Universal Software Validity.

Quality Acceptance Authority

SQS shall identify the competent authority responsible for material quality

acceptance decisions.

Authority may concern:

• Assigned quality responsibility

• Organizational approval

• Delegated acceptance authority

• Defined acceptance procedures

• Applicable contractual conditions

Technical ability to execute tests shall remain distinguishable from

authority to accept quality results.

Quality Assessment Capability ≠ Quality Acceptance Authority.

Quality Acceptance Governance

SQS shall establish whether a material quality acceptance decision is supported by sufficient evidence.

Acceptance may be:

• Accepted

• Conditionally accepted

• Rejected

• Deferred

• Unresolved

Acceptance shall identify its scope, basis, limitations, and competent

authority.

Quality Acceptance ≠ Proof of Defect-Free Software.

Conditional Quality Acceptance

SQS shall recognize quality acceptance subject to explicit limitations or

conditions.

Conditions may include:

• Known non-critical defects

• Restricted usage

• Limited deployment environments

• Required monitoring

• Scheduled corrective actions

• Additional testing requirements

Conditional acceptance shall not be represented as unconditional quality

validation.

Conditional Quality Acceptance ≠ Unrestricted Quality Approval.

Residual Quality Limitations

SQS shall preserve material limitations remaining after quality assessment

or acceptance.

Limitations may include:

• Known defects

• Untested scenarios

• Restricted platform coverage

• Limited workload evidence

• Unverified integrations

• Unresolved behavioral uncertainty

Residual limitations shall remain available for competent downstream

decisions.

Accepted Quality State ≠ Absence of Residual Limitations.

Quality Uncertainty

SQS shall preserve material uncertainty concerning software quality claims.

Uncertainty may arise where:

• Relevant behavior was not tested

• Assessment methods were unsuitable

• Evidence is incomplete

• Artifact correspondence is unresolved

• Configuration correspondence is unclear

• Observations are contradictory

• Defect impact remains uncertain

Uncertainty shall not be silently converted into acceptance.

Unresolved Quality Conditions ≠ Demonstrated Quality Conditions.

Quality Conflict

A Quality Conflict exists where material quality claims or evidence are incompatible.

Examples may include:

• Conflicting test results

• Contradictory performance measurements

• Incompatible defect assessments

• Conflicting artifact references

• Contradictory acceptance records

SQS shall preserve the conflict and its supporting evidence.

Conflicting Quality Evidence ≠ Permission to Select the Preferred Result.

Quality Evidence Gap

A Quality Evidence Gap exists where material information required for competent quality assessment remains insufficiently established.

Examples may include:

• Missing quality criteria

• Incomplete test evidence

• Unknown assessment environment

• Unresolved artifact identity

• Missing defect verification

• Insufficient intended-use evidence

SQS shall identify the gap and its material significance.

Quality Evidence Gap ≠ Proven Software Defect.

Critical Quality Gap

A Critical Quality Gap exists where missing evidence prevents competent assessment of a materially consequential software quality condition.

Examples may include:

• A critical software function without sufficient verification

• A high-impact defect without established resolution evidence

• A release candidate with unresolved artifact-to-test correspondence

• A critical operating condition outside the tested scope

• A software product whose quality acceptance depends on materially

incomplete evidence.

SQS shall preserve the critical gap for competent downstream assessment.

It does not independently determine software shutdown, release rejection, security compromise, or legal liability.

Critical Quality Gap ≠ Automatic Software Failure.

Quality Continuity

SQS shall preserve material quality relationships across software lifecycle

transitions.

Continuity may involve:

• Previous quality assessments

• Artifact changes

• Configuration changes

• Dependency changes

• Defect histories

• Corrective actions

• Regression results

• Acceptance decisions

• Residual limitations

Historical quality evidence shall not automatically establish the quality

of modified software.

Previously Validated Software ≠ Currently Validated Software.

Quality Reassessment

SQS shall support reassessment when material quality conditions change.

Triggers may include:

• New software versions

• Changed configuration

• Dependency updates

• New intended use

• New operating conditions

• Newly identified defects

• Changed quality requirements

• Contradictory assessment results

• New quality evidence

Reassessment shall preserve relevant historical determinations.

Quality Reassessment ≠ Historical Quality Record Erasure.

Software Quality Governance States

SQS shall support identifiable quality governance states.

Possible states include:

Identified: A software quality governance object has been recognized.

Criteria-Defined: Material quality criteria have been established.

Assessment-Pending: Required assessment has not been completed.

Under Assessment: Material quality evaluation is underway.

Evidence-Limited: Available evidence is insufficient for the required determination.

Partially Verified: Defined quality conditions are supported only within a limited scope.

Quality-Verified: Material quality criteria have been sufficiently

demonstrated within the assessed scope.

Validation-Pending: Intended-use validation remains incomplete.

Validated: Material quality conditions are sufficiently supported for the assessed intended use.

Conditionally Accepted: Quality has been accepted subject to explicit

conditions.

Accepted: A competent quality acceptance decision has been established.

Rejected: A competent quality acceptance decision has rejected the

assessed state.

Defect-Unresolved: Material identified defects remain unresolved.

Correction-Pending: Material corrective action remains required.

Conflicted: Material quality claims or evidence are contradictory.

Critical-Gap: A material quality evidence gap prevents competent

assessment.

Superseded: A later quality assessment or software state has replaced the

earlier reference.

Unresolved: Available information does not support a sufficiently determined state.

These states shall remain distinct from software release, deployment,

security, and operational authorization states.

Quality-Verified ≠ Release-Authorized.

Governed Software Quality State

The principal output of SQS is:

Governed Software Quality State

The state shall preserve sufficient information to identify:

Software Quality Governance Object: The assessed software quality object.

Software Identity: The software being evaluated.

Artifact Identity: The applicable artifact or version.

Configuration Reference: The assessed configuration state.

Quality Purpose: The reason for assessment.

Quality Scope: The boundaries of the assessment.

Intended Use: The relevant software purpose and context.

Quality Characteristics: The assessed quality dimensions.

Quality Requirements: The applicable governed requirements.

Quality Criteria: The conditions used for assessment.

Quality Thresholds: Relevant measurable requirements.

Acceptance Conditions: The applicable decision criteria.

Assessment Methods: The methods used.

Method Suitability: The competence of those methods for the relevant

claims.

Test Design: The relevant test definitions.

Test Coverage: The scope of evaluated behavior.

Test Execution: Material execution records.

Assessment Environment: The conditions under which evidence was obtained.

Expected Behavior: The applicable expected software behavior.

Observed Behavior: The actual assessed behavior.

Behavior Correspondence: The relationship between expected and observed

behavior.

Quality Measurements: Relevant measurements and their limitations.

Quality Evidence: Evidence supporting material claims.

Evidence Competence: The suitability of evidence for those claims.

Evidence Integrity: The condition of supporting evidence.

Defect Records: Material identified defects.

Defect Classification: The recognized defect categories.

Defect Severity: The assessed significance of defects.

Defect Impact: The material consequences of defects.

Defect Attribution: Evidence concerning defect causes.

Correction References: Relevant corrective actions.

Resolution Verification: Evidence supporting defect resolution.

Regression Assessment: Relevant post-change quality evidence.

Quality Validation: The scope and result of validation.

Acceptance Authority: The competent decision authority.

Acceptance Decision: The recognized acceptance condition.

Conditional Acceptance: Applicable restrictions and obligations.

Residual Limitations: Known remaining quality constraints.

Quality Uncertainty: Material unresolved conditions.

Quality Conflicts: Contradictory claims or evidence.

Quality Evidence Gaps: Missing material assessment information.

Critical Quality Gaps: Material gaps requiring competent attention.

Quality Continuity: Preservation of relevant lifecycle relationships.

Quality Reassessment: Material reassessment history.

Quality Governance State: The recognized software quality condition.

Downstream References: Information required by competent SLGA™ governance.

The state shall not independently establish software security, release

authorization, deployment authorization, or complete operational suitability.

SQS Core Module Architecture

SQS is implemented through five Core Modules.

Each module governs a distinct quality mechanism while contributing to the

parent standard's principal output. 60.1 QRM — Quality Requirements Module

Governed Mechanism: Software quality requirements, quality

characteristics, criteria, thresholds, and acceptance conditions.

Operational Role: Establish the identifiable and contextually appropriate

conditions against which software quality can be assessed.

Principal Output: Governed Software Quality Requirements State.

Foundational Distinction: Defined Quality Requirements ≠ Demonstrated

Quality. 60.2 QCDM — Quality Criteria Definition Module

Governed Mechanism: The operational definition, interpretation,

measurability, applicability, and evidence requirements of software quality criteria.

Operational Role: Ensure that quality criteria are sufficiently specific, assessable, and traceable to the software object and intended use before they are applied.

Principal Output: Governed Quality Criteria Definition State.

Foundational Distinction: Quality Criterion Declaration ≠ Assessable

Quality Criterion. 60.3 TCMM — Testing & Coverage Management Module

Governed Mechanism: Quality-related test design, execution, coverage,

assessment methods, environment correspondence, and evidence.

Operational Role: Ensure that software testing remains identifiable, appropriately scoped, competently executed, and traceable to the behavior actually evaluated.

Principal Output: Governed Software Testing & Coverage State.

Foundational Distinction: Test Execution Success ≠ Demonstrated Software

Quality. 60.4 DFM — Defect Management Module

Governed Mechanism: Software defect identification, classification,

severity, impact, correction references, resolution verification, and continuity.

Operational Role: Ensure that material defects remain identifiable, evidence-supported, appropriately assessed, and traceable through correction and verification.

Principal Output: Governed Software Defect State.

Foundational Distinction: Defect Closure ≠ Verified Defect Resolution. 60.5 QCAM — Quality Conformity & Acceptance Module

Governed Mechanism: Quality conformity assessment, intended-use

validation, acceptance authority, acceptance decisions, conditional acceptance, residual limitations, and reassessment.

Operational Role: Ensure that quality acceptance remains an explicit, competent, evidence-supported governance determination rather than an automatic consequence of test completion or favorable quality indicators.

Principal Output: Governed Software Quality Conformity & Acceptance State.

Foundational Distinction: Quality Acceptance ≠ Proof of Defect-Free

Software.

SQS Module Governance Flow

The canonical SQS module governance flow is:

Quality Requirements

↓

Quality Criteria Definition

↓

Testing & Coverage Management

↓

Defect Management

↓

Quality Conformity & Acceptance

↓

Governed Software Quality State

The corresponding module flow is:

QRM → QCDM → TCMM → DFM → QCAM

The flow represents methodological dependencies, not a mandatory one-time

linear process.

Quality requirements may require revision when assessment reveals incomplete or unsuitable criteria.

Testing may reveal defects requiring correction.

Corrections may require renewed testing and regression assessment.

Quality acceptance may be conditional, deferred, rejected, or reopened when material evidence changes.

SQS therefore supports iterative quality governance throughout the software

lifecycle.

Governance Requirements

A conforming SQS implementation shall satisfy the following requirements.

SQS-GR-01 — Quality Object Identification

Material quality assessments shall be associated with identifiable software objects and artifacts.

SQS-GR-02 — Quality Purpose Definition

The purpose of material quality assessments shall be established.

SQS-GR-03 — Quality Scope Definition

Assessment boundaries, exclusions, and limitations shall be identifiable.

SQS-GR-04 — Intended-Use Correspondence

Quality determinations shall remain connected to relevant intended use and operating conditions.

SQS-GR-05 — Quality Requirements Correspondence

Material quality criteria shall remain traceable to applicable governed requirements.

SQS-GR-06 — Quality Criteria Establishment

Material quality claims shall be evaluated against sufficiently identifiable criteria.

SQS-GR-07 — Quality Threshold Governance

Relevant measurable thresholds shall preserve their measurement context and limitations.

SQS-GR-08 — Assessment Method Suitability

Assessment methods shall be competent for the quality claims they support.

SQS-GR-09 — Test Design Traceability

Material test cases shall remain associated with relevant quality criteria and assessed software objects.

SQS-GR-10 — Test Coverage Transparency

Coverage claims shall preserve their scope, definition, and limitations.

SQS-GR-11 — Test Execution Evidence

Material test results shall remain associated with actual execution evidence.

SQS-GR-12 — Assessment Environment Correspondence

Quality evidence shall preserve relevant environment and configuration conditions.

SQS-GR-13 — Expected–Observed Behavior Assessment

Material quality claims shall distinguish expected behavior from observed behavior.

SQS-GR-14 — Measurement Interpretation

Quality measurements shall not be generalized beyond the conditions they support.

SQS-GR-15 — Evidence Competence

Material quality evidence shall be suitable for the determination it supports.

SQS-GR-16 — Evidence Integrity

Material quality evidence shall preserve sufficient integrity and traceability.

SQS-GR-17 — Defect Identification

Material defects shall remain identifiable and evidence-supported.

SQS-GR-18 — Defect Severity and Impact Assessment

Material defect significance shall be assessed within the relevant context.

SQS-GR-19 — Defect Resolution Verification

Defect closure shall remain distinguishable from verified resolution.

SQS-GR-20 — Regression Assessment

Material software changes shall trigger appropriate regression assessment where required.

SQS-GR-21 — Quality Validation

Material intended-use quality claims shall remain subject to competent validation.

SQS-GR-22 — Quality Acceptance Authority

Material acceptance decisions shall have an identifiable competent authority.

SQS-GR-23 — Conditional Acceptance Transparency

Conditional quality acceptance shall preserve applicable restrictions and obligations.

SQS-GR-24 — Residual Limitation Preservation

Known material quality limitations shall remain explicit.

SQS-GR-25 — Quality Uncertainty Recognition

Unresolved quality conditions shall not be represented as demonstrated quality.

SQS-GR-26 — Quality Conflict Preservation

Contradictory material quality evidence shall remain identifiable.

SQS-GR-27 — Critical Quality Gap Recognition

Material evidence gaps preventing competent assessment shall be recognized.

SQS-GR-28 — Quality Lifecycle Continuity

Quality determinations shall remain connected to relevant artifact, configuration, and lifecycle changes.

SQS-GR-29 — Quality Reassessment

Material changes to software or assessment conditions shall trigger reassessment where necessary.

SQS-GR-30 — Downstream Quality Handoff

The Governed Software Quality State shall preserve sufficient evidence, uncertainty, and limitations for competent downstream governance.

Governance Logic

The canonical SQS governance logic is:

Receive Governed Software Requirements Baseline

↓

Receive Governed Software Creation State

↓

Receive Governed Software Dependency State

↓

Receive Governed Software Configuration State

↓

Identify Software Quality Governance Object

↓

Establish Quality Purpose and Scope

↓

Identify Intended Use and Assessment Context

↓

Establish Quality Characteristics and Requirements

↓

Define Quality Criteria and Thresholds

↓

Establish Quality Acceptance Conditions

↓

Identify Software Artifact and Configuration References

↓

Select Competent Assessment Methods

↓

Establish Test Design and Coverage Scope

↓

Execute or Evaluate Material Quality Assessments

↓

Preserve Test Execution Evidence

↓

Assess Environment Correspondence

↓

Compare Expected and Observed Software Behavior

↓

Assess Quality Measurements and Their Limitations

↓

Identify and Classify Material Defects

↓

Assess Defect Severity and Impact

↓

Preserve Defect Attribution and Correction References

↓

Verify Material Defect Resolution

↓

Assess Regression Conditions

↓

Associate Quality Evidence

↓

Assess Evidence Competence and Integrity

↓

Identify Quality Uncertainty and Conflicts

↓

Identify Quality Evidence Gaps and Critical Quality Gaps

↓

Validate Material Quality Conditions

↓

Identify Competent Quality Acceptance Authority

↓

Determine Acceptance, Conditional Acceptance, Rejection, or Deferral

↓

Preserve Residual Quality Limitations

↓

Determine Software Quality Governance State

↓

Establish Governed Software Quality State

↓

Handoff to Competent Downstream SLGA™ Governance

↓

Reassess When Material Software or Quality Conditions Change

The principal output is:

Governed Software Quality State

Governance Outputs

SQS may produce:

Software Quality Governance Object

Software Quality Purpose Record

Software Quality Scope Record

Intended-Use Correspondence Record

Software Quality Characteristics Register

Software Quality Requirements State

Quality Criteria Definition State

Quality Threshold Record

Quality Acceptance Conditions Record

Software Artifact Quality Reference

Configuration Quality Correspondence Record

Assessment Method Record

Assessment Method Suitability Assessment

Test Design Record

Test Coverage Record

Test Execution Record

Test Environment Correspondence Assessment

Expected Software Behavior Record

Observed Software Behavior Record

Behavior Correspondence Assessment

Functional Quality Assessment

Reliability Quality Assessment

Performance Quality Assessment

Compatibility Quality Assessment

Usability Quality Assessment

Maintainability Quality Assessment

Portability Quality Assessment

Quality Measurement Record

Quality Measurement Interpretation Record

Quality Evidence Register

Quality Evidence Competence Assessment

Quality Evidence Integrity Assessment

Software Defect Record

Defect Classification Record

Defect Severity Assessment

Defect Impact Assessment

Defect Attribution Record

Defect Correction Reference

Defect Resolution Verification Record

Regression Quality Assessment

Software Quality Validation Record

Quality Acceptance Authority Record

Quality Acceptance Decision

Conditional Quality Acceptance Record

Residual Quality Limitation Record

Quality Uncertainty State

Quality Conflict

Quality Evidence Gap

Critical Quality Gap

Quality Continuity Record

Quality Reassessment Record

Software Quality Governance State

SQS Downstream Governance Handoff Record

The principal output is:

Governed Software Quality State

Critical Distinctions

SQS preserves the following distinctions:

Declared Software Quality ≠ Demonstrated Software Quality

Software Quality Governance Object ≠ Software Test Report

Quality Assessment Purpose ≠ Quality Assessment Result

Tested Software Scope ≠ Complete Software Behavior

Quality Characteristic Selection ≠ Quality Characteristic Satisfaction

Requirement Existence ≠ Demonstrated Requirement Satisfaction

Defined Quality Criterion ≠ Satisfied Quality Criterion

Quality Threshold Achievement ≠ Complete Software Quality

Quality Acceptance Criteria ≠ Quality Acceptance Decision

Technical Functionality ≠ Fitness for Intended Use

Tested Artifact ≠ Every Artifact Bearing the Same Version Label

Tested Configuration ≠ Every Operational Configuration

Assessment Method Execution ≠ Competent Quality Assessment

Successful Assessment Method ≠ Universal Quality Evidence

Defined Test Case ≠ Executed Test Case

High Test Coverage ≠ Demonstrated Software Correctness

Test Execution Completion ≠ Test Success

Test Environment Similarity ≠ Demonstrated Operational Equivalence

Specified Behavior ≠ Observed Behavior

Software Execution ≠ Verified Software Behavior

Observed Expected Output ≠ Complete Behavioral Correctness

Functional Test Success ≠ Complete Functional Suitability

Observed Stability ≠ Guaranteed Future Reliability

Favorable Benchmark Result ≠ Universal Performance Quality

Compatibility with One Environment ≠ Universal Compatibility

Interface Functionality ≠ Demonstrated Usability

Readable Code ≠ Demonstrated Maintainability

Portable Design Intent ≠ Verified Portability

Quality Metric ≠ Software Quality State

Measured Result ≠ Unqualified Quality Conclusion

Quality Evidence Presence ≠ Quality Claim Validity

Evidence of Testing ≠ Evidence of Complete Quality

Intact Quality Report ≠ Verified Software Quality

Reported Software Defect ≠ Verified Software Defect

Defect Classification ≠ Defect Severity

Defect Severity ≠ Defect Remediation Priority

Potential Defect Impact ≠ Observed Defect Impact

Defect Observation ≠ Verified Root Cause

Defect Correction Action ≠ Verified Defect Resolution

Closed Defect Ticket ≠ Verified Defect Resolution

Regression Test Success ≠ Absence of Regression Risk

Validated Quality Scope ≠ Universal Software Validity

Quality Assessment Capability ≠ Quality Acceptance Authority

Quality Acceptance ≠ Proof of Defect-Free Software

Conditional Quality Acceptance ≠ Unrestricted Quality Approval

Accepted Quality State ≠ Absence of Residual Limitations

Unresolved Quality Conditions ≠ Demonstrated Quality Conditions

Conflicting Quality Evidence ≠ Permission to Select the Preferred Result

Quality Evidence Gap ≠ Proven Software Defect

Critical Quality Gap ≠ Automatic Software Failure

Previously Validated Software ≠ Currently Validated Software

Quality Reassessment ≠ Historical Quality Record Erasure

Quality-Verified ≠ Release-Authorized

Cross-Architecture Boundary

SQS governs software quality criteria, assessment, evidence, defects,

validation, and acceptance within SLGA™.

It does not independently establish software requirements, authorize software creation, determine dependency suitability, establish configuration states, certify software security, authorize software releases, approve deployments, or determine legal compliance.

Within SLGA™:

Software Requirements Standard (SRS) governs the requirements against

which quality may be assessed.

Software Creation Standard (SCS) governs software creation processes and

artifacts.

Software Dependency Standard (SDS) governs software dependency identity,

suitability, introduction, and continuity.

Software Configuration Standard (SCFS) governs software configuration

states, builds, artifact versions, configuration changes, and reproducibility.

Software Quality Standard (SQS) governs software quality definition,

criteria, assessment, testing, defects, conformity, and acceptance.

Software Security Standard (SSS) governs software security requirements,

evidence, testing, vulnerabilities, and security risk conditions.

Software Release Standard (SRLS) governs release readiness, authority,

release artifacts, release records, and release decisions.

Software Deployment Standard (SDPS) governs deployment planning,

authority, execution, operational verification, and relevant restoration mechanisms.

Software Change Management Standard (SCMS) governs broader software

lifecycle change processes.

Software Retirement Standard (SRTS) governs software retirement and

relevant continuity obligations.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle oversight, traceability, evidence, and control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

ORA™ — Operational Reality Architecture: Competent assessment of actual

software behavior and operating conditions.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

quality assessment activities and execution boundaries.

AGI™ — Agent Governance Infrastructure: Relevant agent permissions and

tool-path governance.

SQS preserves software-quality-specific governance without replacing these

competent domains.

Canonical Boundary

SQS begins when software quality requirements, criteria, evidence, defects, assessment, validation, or acceptance conditions require governance.

SQS governs:

• Software quality governance objects

• Quality purpose

• Quality scope

• Intended-use correspondence

• Quality characteristics

• Quality requirements correspondence

• Quality criteria

• Quality thresholds

• Quality acceptance conditions

• Artifact correspondence

• Configuration correspondence

• Assessment methods

• Assessment method suitability

• Test design

• Test coverage

• Test execution

• Test environment correspondence

• Expected software behavior

• Observed software behavior

• Expected–observed correspondence

• Functional quality

• Reliability quality

• Performance quality

• Compatibility quality

• Usability and interaction quality

• Maintainability quality

• Portability and adaptability quality

• Quality measurement

• Measurement interpretation

• Quality evidence

• Evidence competence

• Evidence integrity

• Defect identification

• Defect classification

• Defect severity

• Defect impact

• Defect attribution

• Defect correction references

• Defect resolution verification

• Regression assessment

• Quality validation

• Quality acceptance authority

• Quality acceptance

• Conditional quality acceptance

• Residual limitations

• Quality uncertainty

• Quality conflicts

• Quality evidence gaps

• Critical quality gaps

• Quality continuity

• Quality reassessment

• Software quality governance states

SQS does not independently determine:

• Software creation authorization

• Complete dependency suitability

• Configuration baseline establishment

• Build reproducibility

• Complete software security

• Release authorization

• Deployment authorization

• Operational safety certification

• Legal compliance

• Legal liability

SQS ends when material software quality conditions have been represented as a:

Governed Software Quality State

with sufficient quality criteria, assessment methods, observed results,

defect conditions, validation evidence, acceptance decisions, uncertainty, limitations, and continuity information for competent downstream SLGA™ governance.

Canonical Closing Statement

Software quality is not established by declaring that software works.

It is not established merely because tests were executed, code coverage is high, performance benchmarks are favorable, or a quality report has been approved.

A software product may pass every executed test while remaining untested against material real-world conditions.

A defect may be marked as resolved while the underlying failure remains reproducible.

A software artifact may satisfy quality criteria under one configuration while behaving differently under another.

And a competent acceptance decision may recognize residual limitations without establishing that the software is defect-free.

Declared Software Quality ≠ Demonstrated Software Quality.

Test Execution Success ≠ Demonstrated Software Quality.

High Test Coverage ≠ Demonstrated Software Correctness.

Defect Closure ≠ Verified Defect Resolution.

Quality Acceptance ≠ Proof of Defect-Free Software.

These distinctions become increasingly important as software is generated, tested, modified, and evaluated by automated systems and autonomous AI agents.

An AI coding agent may produce functional software.

An automated testing system may report thousands of successful tests.

A quality assessment platform may generate a favorable score.

None of these outcomes independently establishes that the software has been sufficiently evaluated against its intended use, applicable quality criteria, material failure conditions, and relevant operating environment.

SQS establishes the methodological requirement that software quality remain a governed relationship between what software is expected to do, what was actually assessed, what the evidence demonstrates, what remains uncertain, and which quality conditions have been competently accepted.

It does not require software to be perfect.

It requires software quality claims to remain proportionate to the evidence

supporting them.

It does not treat defects as automatic proof of unacceptable software.

It requires defects, their significance, their resolution state, and their

residual implications to remain explicit.

It does not confuse quality acceptance with release authorization.

It provides the governed quality state on which competent release,

deployment, and lifecycle decisions may subsequently rely.

SQS transforms software quality from a collection of test results,

metrics, and declarations into an identifiable, evidence-supported governance state — ensuring that software organizations can establish not merely whether software appears to work, but which quality conditions have been defined, which behaviors have been assessed, what the evidence actually supports, which limitations remain, and under whose authority quality has been accepted.

Software quality is not governed because software passes tests or receives approval. It is governed when its requirements, criteria, assessed behavior, evidence, defects, acceptance conditions, limitations, and lifecycle continuity remain sufficiently established to support competent software lifecycle decisions.

Module Architecture

→ QRM — Quality Requirements Module

→ QCDM — Quality Criteria Definition Module

→ TCMM — Testing & Coverage Management Module

→ DFM — Defect Management Module

→ QCAM — Quality Conformity & Acceptance Module

Related Documents

→ About Software Quality Standard (SQS)